

```
rm(list = ls()) #clear environment to make sure there are no overwriting errors
set.seed(252)
```

The purpose of this document is to compile all of our estimator methods and data (to avoid accidental conflicts/overwriting) and also include statistical inference for these methods. Each section will include the ATE for a particular estimator, as well as statistical inference for that ATE.

Unadjusted Estimator

Estimator

Our unadjusted estimator is simply taking the expected difference in our outcome Y across our two exposure levels, $A = 0, 1$.

```
ATE_unadj <- mean(model_df$Y[model_df$A == 1]) -
              mean(model_df$Y[model_df$A == 0])

ATE_unadj
```

```
[1] 0.4204447
```

Our unadjusted ATE is 0.420. Specifically, the states with more permissive vaccine exemption policies had, on average, about 0.420 additional measles cases per 100,000 people or 4.2 additional measles cases per 10,000 people compared to states with stricter policies. This is a naive estimator, and we will now run the three techniques we learned in class.

Inference

INCLUDE HERE.

G-Computation

We will now perform G-Computation, both with and without using a super learner.

Simple G-Computation

```
#running model
gcomp_mod <- lm(Y ~ A + W1 + W2 + W3 + W4 + W5, data = model_df)

#defining potential outcomes
data_A1 <- model_df
data_A1$A <- 1

data_A0 <- model_df
data_A0$A <- 0

#using model to make predictions
Y1_pred <- predict(gcomp_mod, newdata = data_A1)
Y0_pred <- predict(gcomp_mod, newdata = data_A0)

#defining average treatment effect
ATE <- mean(Y1_pred - Y0_pred)

ATE
```

```
[1] 0.4160642
```

Our adjusted ATE via G-Comp is 0.416. Specifically, the states with more permissive vaccine exemption policies had, on average, about 0.416 additional measles cases per 100,000 people or 4.16 additional measles cases per 10,000 people compared to states with stricter policies. This estimate is very similar to the unadjusted ATE.

G-Computation With Super Learner

```
#defining super learner library
SL.library <- c("SL.mean", "SL.glm", "SL.bayesglm", "SL.earth", "SL.gam")

X <- subset(model_df, select = -Y)
X1 <- X0 <- X
X1$A <- 1
X0$A <- 0
#modeling

gcomp_mod_sl <- SuperLearner(Y = model_df$Y,
```

```

X = X,
SL.library = SL.library,
cvControl = list(V = nrow(model_df)),
family = "gaussian")

#estimating potential outcomes
pred_y1 <- predict(gcomp_mod_sl, newdata = X1)$pred

pred_y0 <- predict(gcomp_mod_sl, newdata = X0)$pred

ATE_SL <- mean(pred_y1 - pred_y0)

ATE_SL

```

```
[1] 0.1549599
```

Statistical Inference

(INCLUDE HERE)

Inverse Probability Weighting

```

# fit SuperLearner for propensity score
SL_propensity <- SuperLearner(
  Y = model_df$A,
  X = subset(model_df, select = -c(A,Y)),
  SL.library = SL.library,
  #cvControl = list(V=10, stratifyCV = TRUE),
  family = binomial()
)

# get predicted probabilities
prob_exposed <- SL_propensity$SL.predict
prob_unexposed <- 1 - prob_exposed

# look for positivity violations
summary(prob_exposed)

```

```
V1
Min.    :0.3333
1st Qu.:0.3333
Median  :0.3333
Mean    :0.3333
3rd Qu.:0.3333
Max.    :0.3333
```

```
summary(prob_unexposed)
```

```
V1
Min.    :0.6667
1st Qu.:0.6667
Median  :0.6667
Mean    :0.6667
3rd Qu.:0.6667
Max.    :0.6667
```

```
# no positivity violations, but all predicted probabilities are pretty close together;
# this indicates the covariates are not very predictive of exposure
```

```
# create weights
```

```
wt_exposed <- as.numeric(model_df$A == 1) / prob_exposed
```

```
wt_unexposed <- as.numeric(model_df$A == 0) / prob_unexposed
```

```
# check weights
```

```
summary(wt_exposed)
```

```
V1
Min.    :0
1st Qu.:0
Median  :0
Mean    :1
3rd Qu.:3
Max.    :3
```

```
summary(wt_unexposed)
```

```

      V1
Min.   :0.0
1st Qu.:0.0
Median :1.5
Mean   :1.0
3rd Qu.:1.5
Max.   :1.5

```

```

# evaluate the IPW estimand
ipw <- mean(wt_exposed*model_df$Y) - mean(wt_unexposed*model_df$Y)
ipw

```

```
[1] 0.4204447
```

```

# evaluate the stabilized IPW estimand
ipw_stabilized <- mean(wt_exposed*model_df$Y)/mean(wt_exposed) - mean(wt_unexposed*model_d
ipw_stabilized      # same as non-stabilized

```

```
[1] 0.4204447
```

Statistical Inference

INCLUDE HERE

TMLE

```

# creating wrapper function for LOOCV

run_ltmle <- function(dat, W=NULL, A, Y, SL.library=NULL, verbose=F){
  set.seed(1)
  df <- dat[,c(W,A,Y)]
  attr(SL.library, "return.fit") <- TRUE
  out <- suppressWarnings(suppressMessages(
    ltmle(data = df, Anodes = A, Ynodes = Y,
          abar = list(1,0),
          SL.library=SL.library,
          SL.cvControl =list(V=nrow(dat)), #leave-one-out CV
          variance.method='ic',

```

```

        estimate.time=F)))
print('Summary of pscore')
print(summary(out$cum.g))

print(summary(out))
# print(out$fit) # provides the TMLE fits (including SL)

out
}

tmle_1 <- run_ltmle(dat = model_df,
                  W = c("W1", "W2", "W3", "W4", "W5"),
                  A = "A",
                  Y = "Y",
                  SL.library=SL.library
                  )

```

[1] "Summary of pscore"

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.3333	0.3333	0.5000	0.5000	0.6667	0.6667

Estimator: tmle

Call:

```

ltmle(data = df, Anodes = A, Ynodes = Y, abar = list(1, 0), SL.library = SL.library,
      SL.cvControl = list(V = nrow(dat)), estimate.time = F, variance.method = "ic")

```

Treatment Estimate:

Parameter Estimate:	1.0706
Estimated Std Err:	0.29989
p-value:	0.00080006
95% Conf Interval:	(0.46822, 1.6729)

Control Estimate:

Parameter Estimate:	0.74027
Estimated Std Err:	0.24085
p-value:	0.0034213
95% Conf Interval:	(0.2565, 1.224)

Additive Treatment Effect:

Parameter Estimate:	0.33031
Estimated Std Err:	0.33416
p-value:	0.32769

95% Conf Interval: (-0.34088, 1.0015)

```
# get estimates & CIs
treatment_est <- summary(tmle_1)$effect.measures$treatment$estimate
control_est <- summary(tmle_1)$effect.measures$control$estimate
control_ci_lower <- summary(tmle_1)$effect.measures$control$CI[1]
control_ci_upper <- summary(tmle_1)$effect.measures$control$CI[2]
treatment_ci_lower <- summary(tmle_1)$effect.measures$treatment$CI[1]
treatment_ci_upper <- summary(tmle_1)$effect.measures$treatment$CI[2]
```

Statistical Inference